ISM2411 — Lab Week 10

Inventory List Manager — Instructor Facilitation Guide

ISM2411 · Python for Business · USF Muma College of Business

Module 10 · Unit 3 · Data Structures

Contents

1 Session Snapshot

Course	ISM2411 — Python for Business
Session	Module 10 Lab — Inventory List Manager
Unit	Unit 3 · Data Structures
Class length	75 minutes
Format	Live code-along
Prerequisites	Modules 05–08: conditionals, loops, functions, Git/GitHub (this module's submission goes through GitHub)
Student-facing lab page	markumreed.github.io/ism2411 — week10_lab
Exercises covered	Exercises 1–8 (required) + Stretch (as time allows)
Submission	`inventory.py` via GitHub (`week10/` folder), repo URL to Canvas

Module 9 was the midterm; this is the first lab of Unit 3, and it opens a genuinely new topic — data structures. Everything before this unit processed values one at a time or accumulated a single running number; this module is about a single variable holding an entire *collection*, and about the specific tools (indexing, slicing, list methods) for reaching into that collection. Two ideas deserve the most protected time: **zero-based indexing** (Exercise 2) and the **mutable-vs-immutable** distinction between lists and tuples (Exercise 5) — both are genuinely non-obvious the first time, and both cause real bugs for the rest of the semester if they don't land solidly today.

2 Learning Objectives

By the end of this 75-minute session, students should be able to:

1. Create a list, and access elements by positive index (from the front) and negative index (from the back), without off-by-one errors.
2. Use slice notation (`[start:stop]`, `[start:]`, `[:stop]`, `[::step]`) to extract a sub-portion of a list.
3. Modify a list in place with `.append()`, `.remove()`, and `.sort()`, and explain why these methods don't need to `return` a new list.
4. Explain why tuples are immutable, and catch the `TypeError` that results from trying to change one.
5. Build a new list from an existing one using the accumulator pattern (`.append()` inside a loop), including a filtered version.

3 Before Class — Setup Checklist

- ☐ Open `inventory.py`, empty except the header comment.
- ☐ Have the exact starting list ready to type identically every time: `inventory = ["pen", "notebook", "stapler", "tape", "marker"]` — five items, used across Exercises 1–4, so index/slice results are consistent and predictable throughout.
- ☐ Pre-draw (on the board, or in a slide) a numbered box diagram of the five-item list showing both positive indices (0–4) and negative indices (-5 to -1) simultaneously — this single visual does more for Exercise 2 than any amount of verbal explanation.
- ☐ Confirm students already have a working GitHub repo from Module 08, since this module's submission format changes (GitHub, not a bare Canvas file upload) — a quick reminder at the start avoids surprise at the end.

4 Materials Needed

- Instructor laptop + terminal + editor, Python 3.10+
- Students: `inventory.py`, GitHub repo from Module 08 with a `week10/` folder to add

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:04	Welcome: "one variable, many values"	4
0:04–0:10	Exercise 1 — Build the list	6
0:10–0:19	Exercise 2 — Index practice	9
0:19–0:27	Exercise 3 — Slicing	8
0:27–0:35	Exercise 4 — Modify	8
0:35–0:45	Exercise 5 — Tuple comparison	10
0:45–0:52	Exercise 6 — Average of a list of sales	7
0:52–1:00	Exercise 7 — Accumulator pattern (build a new list)	8
1:00–1:08	Exercise 8 — Filter above threshold	8
1:08–1:15	Stretch preview + wrap-up, reflection, submission checklist	7

Eight required exercises comfortably fill 75 minutes; the single Stretch challenge (`inventory_report` function) is positioned as a closing preview rather than full live-coded content, since it's primarily a review of Module 07's function skills applied to this module's new list operations, not new material itself.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:04)

Say to the class:

> *"Every variable you've used until now held one value — one price, one total, one customer name. Today, one variable holds a whole* collection*: a list. This opens up indexing (reaching into a specific position), slicing (pulling out a sub-portion), and a handful of methods for changing a list in place. Watch closely for one specific, non-obvious idea: Python counts starting at zero, not one. It genuinely trips up almost everyone the first time."*

Do: Open `inventory.py`, type the header:

```
# inventory.py
# ISM2411 Module 10 Lab – Inventory List Manager
```

6.2 Exercise 1 — Build the List (0:04–0:10, 6 min)

Teaching goal: List literal syntax, and `len()` as the standard way to ask "how many."

Say to the class:

> *"Square brackets, comma-separated values — that's a list. Five strings, one variable."*

Live-code this:

```
# --- Exercise 1 ---
inventory = ["pen", "notebook", "stapler", "tape", "marker"]
print(inventory)
print(len(inventory))
```

Line-by-line explanation:

- `inventory = ["pen", "notebook", "stapler", "tape", "marker"]` — square brackets `[...]` create a **list**; each element is a separate string, comma-separated, in a specific order that Python remembers and preserves — say explicitly: **order matters and is preserved** — this is a real property of lists worth stating up front, since it's what makes indexing (next exercise) meaningful at all.
- `print(inventory)` — printing a list shows it with square brackets and quotes around each string, exactly as it was typed — Python's default way of displaying a list's structure, not something students need to format themselves.
- `len(inventory)` — the same `len()` function from Module 06's `while`-loop exercise (`len(sales)`), now counting list elements instead of being used to bound a loop — worth

connecting explicitly to that prior use.

Run it. Expected output:

```
['pen', 'notebook', 'stapler', 'tape', 'marker']
5
```

Common student mistakes to watch for:

- Using parentheses `(...)` instead of square brackets `[...]` — this actually creates a different thing entirely (a tuple, which Exercise 5 covers), not an error, but a structurally different object; worth a brief "hold that thought, we'll see why this distinction matters in Exercise 5" if it comes up now.
- Forgetting commas between items — produces a `SyntaxError`, or in some cases silently concatenates two adjacent string literals into one (a genuinely obscure Python behavior, `"pen" "notebook"` becomes `"pennotebook"`) — worth a quick live demo only if a student stumbles into it naturally, not worth introducing proactively.

Check for understanding: "If I added a sixth item, what would `len(inventory)` report — and would I need to change anything else in this code for it to be correct?" (`6`, and no — nothing else needs to change; `len()` always reflects the list's current size automatically, which is worth naming as the payoff for using it instead of a hardcoded count.)

6.3 Exercise 2 — Index Practice (0:10–0:19, 9 min)

Teaching goal: Zero-based positive indexing, negative indexing from the end, and computing a "middle" position from `len()` rather than hardcoding it — the single most important non-obvious idea in this lab.

Say to the class:

> *"This is the idea I most want you to leave today with, cold: Python lists are indexed starting at zero. The first item is at position 0, not 1. I want you to say that out loud with me once: 'the first item is at index zero.'"*

Do — point to the board diagram from your setup checklist, mapping both index systems onto the same five items:

```
index:    0        1          2         3       4
value:  "pen"  "notebook"  "stapler"  "tape"  "marker"
neg idx: -5       -4         -3        -2       -1
```

Live-code this:

```
# --- Exercise 2 ---
print(inventory[0])
print(inventory[-1])
mid = len(inventory) // 2
print(inventory[mid])
```

Line-by-line explanation:

- `inventory[0]` — the **first** item, at index `0`. Say explicitly, again: not index `1` — this is the single most common source of off-by-one bugs for the rest of the semester, and it's worth over-emphasizing now rather than assuming it's obvious.
- `inventory[-1]` — **negative indexing**: `-1` means "the last item," counting backward from the end. This is a genuinely convenient Python feature — say explicitly that it avoids needing to compute `len(inventory) - 1` just to reach the last element, which is what you'd have to do in many other languages.
- `mid = len(inventory) // 2` — **floor division**, from Module 04/05, doing real work here: for a 5-item list, `5 // 2` is `2`, and `inventory[2]` is `"stapler"`, the true middle element of an odd-length list. This line is deliberately **not hardcoded** as `inventory[2]` — flag explicitly why: if the list's length changes later, this line still correctly finds the middle, while a hardcoded `2` would silently become wrong.

Run it. Expected output:

```
pen
marker
stapler
```

Common student mistakes to watch for:

- Writing `inventory[1]` expecting the first item — the most common mistake in this entire lab; if you see it, don't just correct the number, have the student recount using the board diagram themselves.
- Writing `inventory[5]` expecting the last item (since there are 5 items) — raises `IndexError: list index out of range`, since valid indices for a 5-item list are only `0` through `4`. This is worth demonstrating live specifically, since it's a very natural mistake to make from ordinary (non-zero-based) counting habits.
- Hardcoding `inventory[2]` for the "middle" instead of computing it from `len()` — technically produces the same output for this specific 5-item list, but defeats the point of the exercise; ask "what happens to this line if I later add a sixth item?" to make the fragility concrete.

Check for understanding: "For a list with 7 items, what index is the true middle, and what expression computes it without hardcoding?" (`7 // 2 = 3`, so `inventory[3]` — get a student to compute this live, not just state the formula, to confirm the reasoning transfers to a different length.)

6.4 Exercise 3 — Slicing (0:19–0:27, 8 min)

Teaching goal: Slice notation for extracting a sub-portion of a list — `[start:stop]`, open-ended slices, and step slices — building directly on Exercise 2's indexing.

Say to the class:

> *"Indexing gets you one item. Slicing gets you a* range *of items — a new, smaller list, cut out of the original. Three flavors today."*

Live-code this:

```
# --- Exercise 3 ---
print(inventory[:3])
print(inventory[-2:])
print(inventory[::2])
```

Line-by-line explanation:

- `inventory[:3]` — a slice with no `start` (defaults to the beginning) and `stop` of `3`: **"everything up to, but not including, index 3."** This is the detail to slow down on: the `stop` value is *exclusive* — `[:3]` gives indices `0, 1, 2`, three items total, not including whatever's at index `3`. Say explicitly: this "stop is exclusive" rule is consistent with `range()` if students have encountered it, and is worth naming as a general Python convention, not a one-off quirk of slicing specifically.
- `inventory[-2:]` — no `stop` (defaults to the end), `start` of `-2`: "starting from the second-to-last item, go to the end" — combines negative indexing with open-ended slicing.
- `inventory[::2]` — **step slicing**: two colons, meaning "no explicit start, no explicit stop, but a step of 2" — take every second element, starting from index `0`. Walk through which indices this actually selects: `0, 2, 4` → `"pen"`, `"stapler"`, `"marker"`.

Run it. Expected output:

```
['pen', 'notebook', 'stapler']
['tape', 'marker']
['pen', 'stapler', 'marker']
```

Common student mistakes to watch for:

- Expecting `inventory[:3]` to include the item at index `3` (`"tape"`) — the exclusive-stop rule is genuinely the most common slicing mistake; have the room count the returned list's length (3 items) against the slice's `stop` value (3) to notice the pattern: **the stop number equals the count of items you get when starting from index 0**, a useful mental shortcut.
- Confusing `[::2]` (step slicing, every second element) with `[:2]` (a plain slice, first two elements) — visually similar, semantically very different; write both side by side if this confusion surfaces.

Check for understanding: "Without running it, what does `inventory[1:4]` return?" (Indices `1, 2, 3` → `['notebook', 'stapler', 'tape']` — three items, again matching the "count = stop − start" shortcut from above, now with an explicit start.)

6.5 Exercise 4 — Modify (0:27–0:35, 8 min)

Teaching goal: List methods that change the list **in place** — `.append()`, `.remove()`, `.sort()` — and the crucial fact that these methods don't `return` a new list; they mutate the original.

Say to the class:

> *"Three methods that change the list itself, permanently, right where it lives in memory — not by giving you back a new, separate list. Print after every single step so you can watch the same variable change shape three times in a row."*

Live-code this:

```
# --- Exercise 4 ---
inventory.append("envelope")
print(inventory)

inventory.remove("tape")
print(inventory)

inventory.sort()
print(inventory)
```

Line-by-line explanation:

- `inventory.append("envelope")` — adds `"envelope"` to the **end** of the list, in place. Note the syntax: `inventory.append(...)`, not `inventory = inventory.append(...)` — say explicitly why the second form is a real trap: `.append()` returns `None`, so assigning its result back to `inventory` would destroy the list, replacing it with `None`. This is worth demonstrating live as a deliberate broken example, since it's a very natural instinct coming from Module 07's "functions return values" lesson — the point to land is that **not every method works like the pure functions from last module; some, like this one, modify their target and hand nothing useful back.**
- `inventory.remove("tape")` — removes the *first* item matching the given value (not a position) — contrast this explicitly with indexing: `.remove()` takes a *value* to search for and delete, not an index to remove by position.
- `inventory.sort()` — sorts the list alphabetically (for strings) **in place**, same "modifies, doesn't return a new list" behavior as `.append()`.

Run it. Expected output (three separate prints, one per step):

```
['pen', 'notebook', 'stapler', 'tape', 'marker', 'envelope']
['pen', 'notebook', 'stapler', 'marker', 'envelope']
['envelope', 'marker', 'notebook', 'pen', 'stapler']
```

Common student mistakes to watch for:

- Writing `inventory = inventory.append("envelope")`, discussed above — demonstrate the `None` result live; a student who does this will find their entire list has vanished on the next `print()`, which is alarming enough to be a genuinely memorable lesson.
- Trying `inventory.remove(0)` expecting to remove "the item at index 0" — this instead searches for the *value* `0` in the list (which isn't present, since this is a list of strings), raising `ValueError: list.remove(x): x not in list`. Contrast explicitly with a hypothetical `del inventory[0]` (removes by position — not covered in this exercise, but worth a one-sentence mention if a curious student asks how to remove by position instead of value).
- Expecting `.sort()` to work on a list containing a mix of types (e.g., if a stray number were in the list) — raises `TypeError`, since Python can't establish an order between fundamentally different types like `str` and `int`; not something to demo unless it comes up naturally, but worth knowing as a quick answer if asked.

Check for understanding: "After these three operations, if I ran `inventory.append("envelope")` a *second* time, what would the list contain?" (Two `"envelope"` entries — lists don't automatically prevent duplicates; `.append()` always adds, regardless of what's already there. Worth confirming this explicitly, since some students assume list-like structures behave like a mathematical set.)

6.6 Exercise 5 — Tuple Comparison (0:35–0:45, 10 min)

Teaching goal: The mutable-vs-immutable distinction between lists and tuples — genuinely the second-most-important idea in this lab, and directly tied to `try`/`except`, a new syntax pattern for gracefully handling an error instead of letting the program crash.

Say to the class:

> *"Everything so far has been a list — changeable, or 'mutable.' Now, a tuple: parentheses instead of square brackets, and fundamentally* unchangeable *once created — 'immutable.' We're going to try to break that rule on purpose, catch the resulting error gracefully instead of crashing, and think about why immutability is actually a* feature*, not a limitation, for something like GPS coordinates."*

Live-code this:

```
# --- Exercise 5 ---
coords = (40.7128, -74.0060)   # New York City

try:
    coords[0] = 99
except TypeError as e:
    print(f"TypeError caught: {e}")
    print("Tuples are immutable — useful for coordinates because a "
          "location's latitude/longitude shouldn't be accidentally "
          "reassignable partway through a program.")

print(coords)
```

Line-by-line explanation:

- `coords = (40.7128, -74.0060)` — parentheses `(...)`, not square brackets, create a **tuple** — syntactically similar to a list, but with a fundamentally different guarantee: once created, its contents cannot be changed, added to, or removed from.
- `try:` / `except TypeError as e:` — **this is new syntax today.** `try` marks a block of code that *might* raise an error; if it does, instead of crashing the whole program, Python jumps to the matching `except` block instead. This is the first time this semester an error is being deliberately *anticipated and handled*, rather than treated purely as a bug to fix before running again. Say explicitly: this is a different relationship with errors than every prior module — sometimes an error is an expected, normal possibility your program should gracefully respond to, not always a mistake to eliminate.
- `coords[0] = 99` — attempting to reassign the first element, exactly like a list assignment would allow — but tuples don't support this at all, regardless of index, raising `TypeError: 'tuple' object does not support item assignment`.
- `except TypeError as e:` — `as e` captures the actual exception object, so its message can

be printed or inspected — `print(f"TypeError caught: {e}")` shows the *exact* error text without crashing.
- The explanation printed inside the `except` block is the exercise's actual required deliverable — a real conceptual point, not just correctly-caught syntax: **coordinates are a natural fit for tuples because a location's latitude and longitude are conceptually a fixed pair that shouldn't drift or be partially edited** — if code elsewhere in a larger program accidentally tried to modify one half of a coordinate pair, immutability turns that mistake into a loud, catchable error instead of a silent, corrupted location.
- `print(coords)` after the `try`/`except` — confirms the tuple is genuinely unchanged, since the assignment inside `try` never actually took effect.

Run it. Expected output:

```
TypeError caught: 'tuple' object does not support item assignment
Tuples are immutable — useful for coordinates because a location's
latitude/longitude shouldn't be accidentally reassignable partway
through a program.
(40.7128, -74.006)
```

(Note the printed tuple shows `-74.006`, not `-74.0060` — Python drops the trailing zero when displaying the float, since `74.0060` and `74.006` are the exact same numeric value; worth a ten-second aside if a sharp student notices the discrepancy.)

Common student mistakes to watch for:

- Omitting the `try`/`except` entirely and just running `coords[0] = 99` directly — the program crashes with an unhandled `TypeError` and stops; a good moment to run it both ways (with and without the `try`) side by side, so the room sees exactly what `try`/`except` is buying them: the program *continues running* past the error instead of stopping dead.
- Catching a broader exception type than needed (e.g., a bare `except:` with no type specified) — works for this exercise, but worth a brief caution that catching *any and all* errors indiscriminately can hide real bugs you didn't anticipate; catching the *specific* expected error type (`TypeError` here) is the more disciplined habit.
- Confusing "immutable" with "can't be printed" or "can't be read" — tuples can absolutely still be read/accessed/printed freely; only *reassignment* of an element is disallowed. Worth stating this distinction explicitly, since "immutable" is a genuinely new vocabulary word this module.

Check for understanding: "If tuples can't be changed, why not just always use lists for everything, and skip tuples entirely?" (A good answer names at least one of: immutability as a safety guarantee against accidental modification, e.g. for something that should behave like a fixed record; or the fact that tuples signal *intent* — "this collection of values is meant to stay fixed" — to anyone reading the code later, which lists don't communicate.)

6.7 Exercise 6 — Average of a List of Sales (0:45–0:52, 7 min)

Teaching goal: Python's built-in `sum()`, `max()`, and `min()` functions — directly contrasted with Module 06's manually-built accumulator versions of the exact same computations.

Say to the class:

> *"Remember Module 6, where you built sum, max, and min by hand with a loop and an accumulator? Python has built-in functions that do all of that in one call. I want you to genuinely notice both how much shorter this is,* and *why it was still worth learning to build them by hand first."*

Live-code this:

```
# --- Exercise 6 ---
sales = [100, 250, 75, 480, 200]

total = sum(sales)
count = len(sales)
average = total / count

print(f"Total: {total} | Average: {average} | Best: {max(sales)} | Worst: {min(sales)}")
```

Line-by-line explanation:

- `sum(sales)` — Module 06 Exercise 1's entire accumulator loop, replaced by a single built-in call. Say explicitly: this is the exact computation students built by hand with `total = 0; for sale in sales: total += sale` — now available as one function call, because summing a list is common enough that Python provides it directly.
- `max(sales)` / `min(sales)` — same relationship to Module 06 Exercise 3's hand-built max-tracking loop (`current_max = 0; for sale in sales: if sale > current_max: current_max = sale`).
- **The point worth making explicitly, since some students will (reasonably) ask "why did we do it the hard way first?":** understanding *how* `sum()`/`max()`/`min()` work internally — because you built the equivalent loop by hand — means you're not just trusting a black box. It also means when you need something slightly different that no built-in covers exactly (like Exercise 7's "build a new list with every value discounted," which has no single built-in function), you already know the pattern to reach for.

Run it. Expected output:

```
Total: 1105 | Average: 221.0 | Best: 480 | Worst: 75
```

Common student mistakes to watch for:

- Calling `sum` or `max`/`min` without parentheses, or on the wrong variable — low-risk mistakes at

this point in the semester, but worth a quick visual check as you circulate.

- Not noticing that `average` here prints as `221.0`, not `$221.00` — this exercise's expected output doesn't use currency formatting; if a student adds `${average:.2f}` on their own initiative, that's not wrong, just going beyond what's asked — worth a quick "good instinct, not required here" acknowledgment rather than correcting it.

Check for understanding: "If `sales` had a negative number in it (say, a returned/refunded day), would `min()` still work correctly, and would today's Exercise 2 initialization pattern (`current_max = 0`) have?" (`min()`/`max()` on the built-in functions handle negative numbers correctly with no special handling needed — a good callback to Module 06's flagged limitation of initializing a hand-built accumulator to `0`, which *would* break with negative values; built-ins don't have that limitation because they don't need an initial guess at all.)

6.8 Exercise 7 — Accumulator Pattern: Build a New List (0:52–1:00, 8 min)

Teaching goal: Use `.append()` inside a loop to build an entirely new list from an existing one — a new *shape* of accumulator pattern: instead of accumulating into a single number, accumulate into a growing list.

Say to the class:

> *"New variation on the accumulator pattern from Module 6: instead of building up one number, we're building up an entire new list, one `.append()` at a time."*

Live-code this:

```
# --- Exercise 7 ---
prices = [9.99, 14.99, 4.99, 24.99, 1.99]
discounted = []                        # initialize — an empty list this time
for price in prices:
    discounted.append(round(price * 0.9, 2))   # update — grow the list

print(prices)
print(discounted)
```

Line-by-line explanation:

- `discounted = []` — **initialize**, exactly like Module 06's `total = 0`, but here the "empty" starting value is an empty list, not zero — say explicitly: the *shape* of what you initialize should match the shape of what you're building; a running number starts at `0`, a growing list starts at `[]`.
- `discounted.append(round(price * 0.9, 2))` — **update**, inside the loop: for each `price`, compute the 10%-off value (`price * 0.9`), round it to 2 decimal places with the built-in `round()` function, and append the result onto the end of `discounted`. Introduce `round()` explicitly here as new: it's not an f-string format spec this time (which only affects *display*) — `round()` actually changes the *stored* numeric value, which matters because this value is going into a list, not being printed directly with a format spec.
- The original `prices` list is completely untouched by this loop — say explicitly: this is a deliberate design choice, building a *new* list rather than modifying `prices` in place, which is worth contrasting with Exercise 4's `.append()`/`.remove()`/`.sort()`, all of which *did* modify their list directly. Ask the room: "why might we want to keep the original prices list intact here, rather than overwriting it with discounted values?" (Because you likely need both — the original price for a receipt or comparison, and the discounted price for the actual charge; overwriting would destroy information you still need.)

Run it. Expected output:

```
[9.99, 14.99, 4.99, 24.99, 1.99]
```

```
[8.99, 13.49, 4.49, 22.49, 1.79]
```

Common student mistakes to watch for:

- Forgetting `round(..., 2)` — without it, floating-point arithmetic can produce ugly, long decimal values (e.g., `4.99 * 0.9` is `4.4909999999999997` in raw floating point) rather than a clean `4.49`; run this live without `round()` to show the room exactly why it's needed here, not just told to include it.
- Trying to build the new list by directly modifying `prices` in the loop (e.g., `prices[i] = ...`) instead of appending to a separate `discounted` list — this would require index-based iteration (not covered by this exercise's simple `for price in prices:` pattern) and, more importantly, destroys the original data the exercise explicitly asks to preserve.
- Initializing `discounted` inside the loop instead of before it — same category of mistake as Module 06's central lesson, now applied to a list instead of a number: this would reset `discounted` to `[]` on every pass, ending with a list containing only the *last* item's discounted price.

Check for understanding: "How many times does `.append()` get called in total when this loop finishes, and how do you know?" (Five times — once per item in `prices`, since the loop runs once per element and `.append()` is called unconditionally on every pass, with no filtering `if` involved yet — that's coming next, in Exercise 8.)

6.9 Exercise 8 — Filter Above Threshold (1:00–1:08, 8 min)

Teaching goal: Combine Exercise 7's "build a new list" pattern with a filtering `if` — the list-building equivalent of Module 06 Exercise 4's filtered accumulator, now producing a genuinely new, shorter list instead of just a count and a total.

Say to the class:

> *"Same 'build a new list' pattern as Exercise 7, but now with a filter — only sales days over $300 make it into the new list at all."*

Live-code this:

```
# --- Exercise 8 ---
sales = [340, 127, 589, 204, 467, 88, 731, 315, 62, 490]
high_sales = []                    # initialize
for sale in sales:
    if sale > 300:                  # filter
        high_sales.append(sale)     # update, conditionally

print(high_sales)
```

```
print(len(high_sales))
print(sum(high_sales) / len(high_sales))
```

Line-by-line explanation:

- `high_sales = []` — initialize, same pattern as Exercise 7.
- `if sale > 300: high_sales.append(sale)` — the `.append()` is now **inside an `if`**, exactly like Module 06 Exercise 4's `count_over_100 += 1` was nested inside a filter — the same double-indentation to watch for: the `if` is one level inside the loop, `.append()` is a second level inside the `if`.
- `sum(high_sales) / len(high_sales)` — reuses Exercise 6's built-in functions, now operating on the *filtered* list rather than the original — a good moment to confirm the room understands `high_sales` is a genuinely separate, real list at this point, not just a filtered "view" of `sales` — you could modify `high_sales` from here on without touching the original `sales` list at all.

Run it. Expected output:

```
[340, 589, 467, 731, 315, 490]
6
488.6666666666667
```

Common student mistakes to watch for:

- Using >= instead of > for the $300 threshold — the lab's spec says "exceeded 300," meaning strictly greater; worth a quick check that no sale exactly equal to `300` exists in this dataset anyway (there isn't one here, so this particular mistake wouldn't actually change today's output — but flag that it *would* matter with different data, echoing Module 05's repeated emphasis on reading boundary language carefully).
- Appending `True`/`False` (the result of the comparison) instead of `sale` (the actual value) — a copy-paste-style mistake, e.g. `high_sales.append(sale > 300)` — produces a list of booleans instead of sale amounts; a good "does this list even make sense" sanity check if it happens, since eyeballing `[True, True, True, ...]` should look obviously wrong against the exercise's stated goal.

Check for understanding: "How is this exercise structurally identical to Module 6 Exercise 4's filtered count, and how is it genuinely different?" (Identical: both use an accumulator initialized before a loop, updated conditionally inside an `if`. Different: Module 06 Exercise 4 accumulated a *count* and a *running total* — two numbers; this exercise accumulates an entire *new list* of the qualifying values themselves, which is strictly more information — from a resulting list you can always compute a count or total afterward, but from just a count and total you can't reconstruct which specific values contributed.)

6.10 Stretch — `inventory_report` Function (1:08–1:15, as time allows)

Frame as a quick preview/demo if time is short — this is primarily Module 07 review (writing a function, calling it more than once with different inputs) applied to this module's new list vocabulary, not new material:

```
def inventory_report(items):
    print(f"Count: {len(items)}")
    print(f"First (alphabetically): {min(items)}")
    print(f"Last (alphabetically): {max(items)}")
    print(f"Sorted: {sorted(items)}")

inventory_report(inventory)
inventory_report(["zebra", "apple", "mango"])
```

Two things worth saying explicitly if you demo this live:

- `min(items)` / `max(items)` on a list of **strings** returns the alphabetically first/last item, not a numeric minimum/maximum — worth stating explicitly as a nice generalization of Exercise 6's numeric `min()`/`max()`: these functions work on any type that can be meaningfully compared/ordered, strings included.
- `sorted(items)` (a **function**, called as `sorted(items)`) versus Exercise 4's `.sort()` (a **method**, called as `inventory.sort()`) — this is a genuinely useful distinction worth naming if time allows: `sorted()` returns a *new*, sorted list and leaves the original untouched, while `.sort()` modifies the original list in place and returns nothing. Calling `inventory_report` a second time, on a completely different list, is what confirms the function genuinely generalizes rather than being hardcoded to the specific `inventory` list from earlier in the file.

7 Wrap-Up (last ~7 minutes)

Review the reflection questions out loud (these are answered as a comment at the top of the file, per this lab's submission format — different from prior modules' end-of-script reflection placement, worth noting explicitly):

1. *Most surprising thing about how lists work* — no wrong answer; common genuine surprises include zero-based indexing, negative indexing, or the fact that `.append()`/`.sort()` modify in place rather than returning a new list.
2. *A real business situation where a list would have saved time vs. Excel* — push for specificity: which repeated manual task, what would the list have contained, and what operation (filtering, sorting, summing) would have replaced manual work.

Review the submission checklist together:

- ☐ File is named `inventory.py`
- ☐ Contains Exercises 1–8, each clearly separated
- ☐ Reflection comment at the top of the file, answering both questions
- ☐ Pushed to GitHub inside a `week10/` folder
- ☐ Repo URL submitted to Canvas

Preview Module 11: "Lists gave you an ordered collection, reached by position. Next module, dictionaries give you a collection reached by *name* — a customer's data organized as `name`, `email`, `tier`, not position `0`, `1`, `2`. This is the structure behind every real CRM and analytics system you'll ever touch professionally."

8 Appendix A — Full Answer Key (`inventory.py`)

```
# inventory.py
# ISM2411 Module 10 Lab – Inventory List Manager
# Reflection:
# 1. [Most surprising thing about lists – student's own words]
# 2. [Real business situation where a list would have saved time]

# --- Exercise 1 ---
inventory = ["pen", "notebook", "stapler", "tape", "marker"]
print(inventory)
print(len(inventory))

# --- Exercise 2 ---
print(inventory[0])
print(inventory[-1])
```

```python
mid = len(inventory) // 2
print(inventory[mid])

# --- Exercise 3 ---
print(inventory[:3])
print(inventory[-2:])
print(inventory[::2])

# --- Exercise 4 ---
inventory.append("envelope")
print(inventory)
inventory.remove("tape")
print(inventory)
inventory.sort()
print(inventory)

# --- Exercise 5 ---
coords = (40.7128, -74.0060)
try:
    coords[0] = 99
except TypeError as e:
    print(f"TypeError caught: {e}")
    print("Tuples are immutable — useful for coordinates because a "
          "location's latitude/longitude shouldn't be accidentally "
          "reassignable partway through a program.")
print(coords)

# --- Exercise 6 ---
sales = [100, 250, 75, 480, 200]
total = sum(sales)
count = len(sales)
average = total / count
print(f"Total: {total} | Average: {average} | Best: {max(sales)} | Worst: {min(sales)}")

# --- Exercise 7 ---
prices = [9.99, 14.99, 4.99, 24.99, 1.99]
discounted = []
for price in prices:
    discounted.append(round(price * 0.9, 2))
print(prices)
print(discounted)
```

```python
# --- Exercise 8 ---
sales = [340, 127, 589, 204, 467, 88, 731, 315, 62, 490]
high_sales = []
for sale in sales:
    if sale > 300:
        high_sales.append(sale)
print(high_sales)
print(len(high_sales))
print(sum(high_sales) / len(high_sales))
```

Stretch (`inventory_report` function):

```python
def inventory_report(items):
    print(f"Count: {len(items)}")
    print(f"First (alphabetically): {min(items)}")
    print(f"Last (alphabetically): {max(items)}")
    print(f"Sorted: {sorted(items)}")

inventory_report(inventory)
inventory_report(["zebra", "apple", "mango"])
```

9 Appendix B — Extra Practice (only if the class finishes early)

Eight required exercises comfortably fill the full 75 minutes at a normal pace. If a section moves unusually fast:

Extra — a second indexing/slicing round, different list. `colors = ["red", "orange", "yellow", "green", "blue", "indigo", "violet"]`. Have students independently print: the first item, the last item, the true middle item (computed from `len()`), the first four items via slice, the last three via a negative slice, and every third item via `[::3]`. (First: `red`. Last: `violet`. Middle (`7 // 2 = 3`): `green`. First four: `['red', 'orange', 'yellow', 'green']`. Last three: `['blue', 'indigo', 'violet']`. Every third: `['red', 'green', 'violet']`.)

Extra — a second filtered-list-build, different data. `temps = [58, 72, 91, 45, 88, 63, 95, 70]`. Build a new list `hot_days` containing only temperatures at or above 85, then print the list, its count, and the average. (`hot_days = [91, 88, 95]`. Count: `3`. Average: `91.33`.)